

DESCRIPTION D'UNE MISSION | BTS SIO

Prénom – Nom	RAPHEHISON Mandresy
N° mission	E-06

Option	☐ SISR	☒ SLAM
Situation	☐ Formation	☒ Entreprise

Lieu de réalisation	Carrefour CSI – 93 Avenue de Paris, 91300 Massy	
Période de réalisation	Du : 07/05/2026 Au : 27/05/2026	
Modalité de réalisation	☒ VÉCUE ☐ OBSERVÉE	

Intitulé de la mission	Ajout d'un sélecteur de mode d'exécution des checkers HTTP dans l'interface Synthetic
Description du contexte	L'application Synthetic de LOGM permet aux équipes de créer et gérer des testsuites de monitoring HTTP et Browser. Jusqu'alors, l'exécution des tests était limitée à un seul checker HTTP à la fois. Afin d'augmenter la flexibilité et de couvrir des scénarios plus avancés (tests de résilience, comparaison de réponses entre régions), il a été décidé d'introduire un sélecteur de mode d'exécution des checkers HTTP.

Ressources et outils utilisés	- React (hooks : useState, useEffect) - Redux (état global de l'application) - Material UI (composants Collapse, RadioGroup, FormControlLabel, Icons) - TypeScript - VS Code / environnement de développement Carrefour - Ticket Jira décrivant le besoin et les critères d'acceptation
Résultat attendu	- Un sélecteur de mode d'exécution (ALL / RANDOM / SEQUENTIAL) visible uniquement lorsque plusieurs checkers HTTP sont sélectionnés dans la liste - La valeur sélectionnée est stockée dans l'état du composant (checkerExecutionMode) et transmise au backend lors de la sauvegarde de la testsuite - La propriété executionMode est ajoutée au payload envoyé à l'API
Contraintes	- S'intégrer dans le composant AddEditTest.tsx existant sans casser l'existant - Le sélecteur ne doit apparaître que si plus d'un checker est sélectionné (Collapse) - Respecter les conventions de code et les composants MUI déjà utilisés - Mission en cours : la logique d'exécution côté front n'est pas encore implémentée

Compétences associées (voir tableau)	- B2.1 – Concevoir et développer une solution applicative - B2.2 – Assurer la maintenance corrective ou évolutive d'une solution applicative
---	---

Description des étapes de réalisation (démarche, méthodes, techniques utilisées)

1. ANALYSE DU BESOIN ET DE L'EXISTANT

La mission a débuté par la lecture du ticket Jira décrivant le besoin : permettre à l'utilisateur de choisir parmi trois modes d'exécution (ALL, RANDOM, SEQUENTIAL) lorsque plusieurs checkers HTTP sont sélectionnés. Le critère principal était que le sélecteur ne soit visible que dans ce cas précis.

J'ai ensuite analysé le composant AddEditTest.tsx existant pour comprendre comment le sélecteur de checkers HTTP était déjà géré. J'ai identifié :

- Le state externalChecker (string, checkers séparés par des virgules)
- Le composant Collapse déjà utilisé pour afficher des options conditionnelles
- La structure du payload envoyé à l'API lors de la sauvegarde (handleSubmitSave,

```
handleSubmitUpdateAndClose, handleSubmitUpdateOnly)
```

2. AJOUT DU STATE ET DU SÉLECTEUR

J'ai ajouté un nouvel état React pour stocker le mode d'exécution sélectionné :

```
const [checkerExecutionMode, setCheckerExecutionMode] = useState<
  'ALL' | 'RANDOM' | 'SEQUENTIAL'>('ALL');
```

Le sélecteur a été intégré dans un composant Collapse existant, conditionné par le nombre de checkers sélectionnés :

```
in={externalChecker.split(',').filter(c => c.trim() !== "").length > 1}
```

À l'intérieur du Collapse, j'ai utilisé un RadioGroup MUI avec trois options représentées par des FormControlLabel, chacune accompagnée d'une icône explicite :

- ALL → icône DoneAllIcon
- RANDOM → icône ShuffleIcon
- SEQUENTIAL → icône RepeatIcon

Un composant LogmHelp (bulle d'aide interne) a été ajouté pour expliquer chaque mode.

3. INTÉGRATION AU PAYLOAD API

La valeur de checkerExecutionMode est incluse dans l'objet checkerSettings envoyé à l'API lors de chaque sauvegarde ou mise à jour :

```
checkerSettings: {
  allCheckersFailed,
  executionMode: checkerExecutionMode,
}
```

Lors du chargement d'une testsuite existante (mode édition), la valeur est récupérée depuis testEdit.checkerSettings.executionMode et normalisée en majuscules pour garantir la compatibilité avec les valeurs attendues.

4. RÉINITIALISATION AUTOMATIQUE

Un useEffect a été ajouté pour réinitialiser automatiquement le mode sur 'ALL' si l'utilisateur revient à un seul checker sélectionné, évitant ainsi un état incohérent où un mode multi-checker serait conservé sans checkers multiples.

5. CE QUI RESTE À FAIRE

La logique d'exécution côté front n'est pas encore implémentée : selon le mode sélectionné, le comportement lors du lancement des tests devra être adapté (sélection aléatoire d'un checker, rotation séquentielle ou exécution parallèle). Cette partie fera l'objet d'une prochaine itération.

Conclusion	Cette mission m'a permis de travailler sur l'évolution d'un composant React complexe déjà en production, en y intégrant une nouvelle fonctionnalité sans perturber l'existant. L'analyse préalable du code existant (état, payload, Collapse) a été essentielle pour s'intégrer proprement dans l'architecture en place. L'utilisation des composants MUI (RadioGroup, Collapse, Icons) et des conventions TypeScript du projet m'a permis de produire un code cohérent avec le reste de l'application. La mission étant en cours, elle illustre également la capacité à livrer des fonctionnalités de manière itérative.
Productions associées	- Composant AddEditTest.tsx modifié : ajout du state checkerExecutionMode, du sélecteur RadioGroup conditionnel et de l'intégration au payload API - Code versionné sur le dépôt GitLab interne LOGM (commit en attente)